

Descubrimiento P2P

Gossip + DHT Chord simplificado

Claudio Omar Biale

Agenda

1. **Gossip**: propagación epidémica de estado
2. **DHT Chord**: lookup distribuido en anillo
3. **Ejemplos**: ejemplos de aplicaciones usando Gossip y Chord simplificado
4. **Proyecto**: sd-descubrimiento (PG 3)

PARTE I: Gossip

Gossip: concepto

- **Idea:** cada nodo habla periódicamente con un par aleatorio e intercambia información
- **Push vs Pull:** Push (enviar datos propios), Pull (solicitar datos ajenos), o Push-Pull (ambos)
- **Resultado:** cualquier estado nuevo llega a todos en aproximadamente $O(\log N)$ rondas
- **Usos:** membresía de cluster, métricas, propagación de config

"Como los rumores en una fiesta: todos se enteran sin que nadie coordine"

Anti-entropía

- **Anti-entropía**: intercambio completo de estado entre dos nodos.
 - Nodo A envía **todos** sus miembros conocidos al nodo B.
 - Nodo B responde con los suyos.
 - Ambos fusionan: `union = A U B`
- **Rumor mongering** (variante): solo se envían las **novedades** (estado recién aprendido), no todo el conjunto.
- **Analogía**: como sincronizar dos directorios con `rsync` — se copia todo, no solo los cambios.

Cassandra combina ambas: anti-entropía periódica + rumor mongering entre ciclos.

Estructura NodoGossip

```
type NodoGossip struct {
    ID          int           // Identificador único del nodo
    Direccion   string        // Dirección `:puerto` donde escucha RPC
    Miembros    map[string]bool // Tabla de membresía: `"addr:puerto"` → `true` si vivo
    mu          sync.RWMutex    // Protege `Miembros` de lecturas/escrituras concurrentes
}

func (n *NodoGossip) AntiEntropia() string {
    n.mu.RLock()
    defer n.mu.RUnlock()
    for dir := range n.Miembros {
        if dir != n.Direccion {
            return dir
        }
    }
    return ""
}
```

Checkpoint: `Miembros` contiene todos los nodos conocidos y es tanto el estado que se gossipea como la fuente de pares. Al arrancar, una semilla inicial provee la primera lista de miembros.

Evolución de la membresía por rondas

Ronda	Nodo 1	Nodo 2	Nodo 3	Nodo 4	Nodo 5	Intercambios
0	{1}	{2}	{3}	{4}	{5}	(inicio)
1	{1,3}	{2,5}	{1,3}	{4}	{2,5}	1 ↔ 3, 2 ↔ 5
2	{1,2,3,4,5}	{1,2,3,5}	{1,2,3,5}	{1,2,3,4,5}	{1,2,3,5}	1 ↔ 2, 3 ↔ 5, 4 ↔ 1
3	✓ completo	✓ completo	✓ completo	✓ completo	{1,2,3,5}	1 ↔ 2, 4 ↔ 3
4	✓ completo	✓ completo	✓ completo	✓ completo	✓ completo	5 ↔ 1

- **Ronda 1:** pares 1 ↔ 3 y 2 ↔ 5 intercambian. Nodo 4 aislado.
- **Ronda 2:** 1 ↔ 2, 3 ↔ 5 y 4 ↔ 1 intercambian; el 4 descubre a todos vía Nodo 1.
- **Ronda 3:** 1 ↔ 2 y 4 ↔ 3; los nodos 2 y 3 obtienen la clave 4. Nodo 5 aún no converge.
- **Ronda 4:** 5 ↔ 1; el nodo 5 obtiene la clave 4. Convergencia total.

Checkpoint Gossip

Pregunta: Si tenemos N nodos y cada uno elige 1 par aleatorio por ronda, ¿cuántas rondas (aproximadamente) hasta convergencia?

Respuesta: $O(\log N)$ rondas. Para 5 nodos, ~3-4 rondas. Para 1000 nodos, ~10 rondas.

PARTE II: DHT Chord

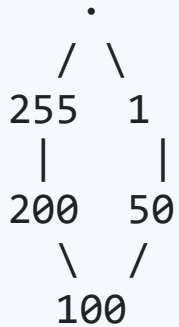
DHT: concepto

- **Tabla Hash Distribuida:** almacenar clave-valor sin servidor central
- **Anillo lógico:** IDs de 0 a 255 (8 bits)
- **Responsabilidad:** cada nodo guarda las claves en (predecesor, nodo]

"Como un archivo pero circular: cada uno se encarga de un rango"

Chord simplificado

Anillo 0-255 (nodos 1, 50, 100, 200, 255 = sin nodo):



- Nodo 1: responsable de $(255, 1]$ → claves $\{0, 1\}$
- Nodo 50: responsable de $(1, 50]$ → claves $\{2..50\}$
- Nodo 100: responsable de $(50, 100]$ → claves $\{51..100\}$
- Nodo 200: responsable de $(100, 200]$ → claves $\{101..200\}$
- Nodo 255: responsable de $(200, 255]$ → claves $\{201..255\}$

Finger Table (3 entradas)

Nodo 1:

Entrada 0: sucesor de $(1 + 2^0) = \text{sucesor de } 2 \rightarrow \text{apunta a nodo } 50$
Entrada 1: sucesor de $(1 + 2^1) = \text{sucesor de } 3 \rightarrow \text{apunta a nodo } 50$
Entrada 2: sucesor de $(1 + 2^2) = \text{sucesor de } 5 \rightarrow \text{apunta a nodo } 50$

- En el libro la formula es: $FT[i] = \text{succ}(p + 2^{(i-1)})$ pero i comienza en 1, en este ejemplo usamos indexado de i desde 0.
- Chord realmente usa s entradas donde s es menor o igual a m , siendo m igual a la cantidad de bits de los IDs.

Lookup: buscar clave 75 desde nodo 1

Nodo 1 (pred=255): ¿75 en (255, 1]? No.

Mejor salto en finger table: sucesor 50. Forward.

Nodo 50 (pred=1): ¿75 en (1, 50]? No.

Mejor salto en finger table: sucesor 100. Forward.

Nodo 100 (pred=50): ¿75 en (50, 100]? Sí. Soy responsable.

Resultado: clave 75 → Nodo 100 (2 hops)

Estructura NodoChord

```
type NodoChord struct {  
    ID          int           // Posición del nodo en el anillo (0-255)  
    Direccion   string        // Dirección `:puerto` donde escucha RPC  
    Sucesor     string        // Dirección del siguiente nodo en el anillo  
    Predecesor  string        // Dirección del nodo anterior en el anillo  
    FingerTable [3]string     // 3 entradas para saltos rápidos (simplificado)  
    Datos       map[int]string // Pares clave-valor que este nodo almacena  
}
```

Método	¿Qué hace?
EsResponsable(clave)	¿Esta clave cae en el rango (predecesor, ID] de este nodo?
MejorSalto(clave)	Busca en la FingerTable el nodo más cercano a la clave destino

Checkpoint DHT

Ejercicio: Tracear la búsqueda de la clave 200 desde el Nodo 50.

- Nodo 50 (pred=1): ¿200 en (1, 50]? No. Mejor salto : sucesor 100. Forward.
- Nodo 100 (pred=50): ¿200 en (50, 100]? No. Mejor salto : sucesor 200. Forward.
- Nodo 200 (pred=100): ¿200 en (100, 200]? **Sí**. Soy responsable.

Hops: 2

Proyecto: sd-descubrimiento (PG 3)

Objetivo: Nodo que ejecuta Gossip + DHT simultáneamente.

Endpoints HTTP:

- GET /estado → membresía + finger table
- POST /almacenar → guardar clave-valor
- GET /buscar?clave=X → lookup

RPC:

- ServicioGossip.Intercambiar → anti-entropía
- ServicioGossip.Identificarse → dirección + ID canónico (estabilización Chord, resuelto)
- ServicioChord.Almacenar → almacenar con forwarding via finger table
- ServicioChord.Obtener → lookup con forwarding via finger table

Preguntas

Descubrimiento P2P